kssanthoshkumar / capstone_project
Code · Issues · Pull requests · Agents · Actions · Projects · Security and quality · Insights
Type / to search
Files
main
Go to file
.github
data
models
notebooks
presentations
reports
src
tests
.env.example
.gitignore
DEPLOYMENT.md
Dockerfile
LICENSE
README.md
network_anomaly_detection_ui_d...
requirements.txt
train_and_save.py
capstone_project / DEPLOYMENT.md
kssanthoshkumar Adding UI Screen updated the deployment, also gen ai using local llm ... 476f930 · 16 minutes
Preview Code Blame 383 lines (285 loc) · 11.8 KB Raw

Deployment Guide — Network Anomaly Detection

This guide covers local deployment of the FastAPI inference API and the Streamlit UI, the retraining pipeline, and the MLOps practices adopted in this project.

Prerequisites

Requirement	Version	Notes
Python	≥ 3.11	Tested on 3.11 and 3.12
pip	≥ 23	Bundled with Python
RAM	≥ 4 GB	Required for XGBoost inference
Disk	≥ 500 MB	Data + model artefacts

---

1. Installation

```
# Clone the repository
git clone https://github.com/[your-username]/network-anomaly-detection.git
cd network-anomaly-detection

# Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate          # macOS / Linux
# .venv\Scripts\activate           # Windows

# Install all dependencies
pip install -r requirements.txt
```

---

2. Train the Model (first-time setup)

The training script downloads NSL-KDD automatically if the raw files are absent, trains XGBoost, and saves all artefacts to `models/`.

```
python train_and_save.py
```

Expected output:

```
[1/3] Loading NSL-KDD dataset...
      Train: (125973, 45)  |  Test: (22544, 45)
[2/3] Preprocessing...
[3/3] Training XGBoost (GridSearchCV, 3-fold CV)...
      Test F1=0.776  AUC-ROC=0.967  (KDDTest+ held-out set)
✅  Training complete. Artifacts saved to models/
    Start the API with:  uvicorn src.app:app --port 8000
```

Saved artefacts:

File	Description
`models/xgboost.pkl`	Best model (XGBoost)
`models/preprocessor.pkl`	Fitted ColumnTransformer
`models/configs.yaml`	Hyperparameters + metadata
`data/processed/*.npy`	Pre-split arrays for notebooks

---

3. Running the FastAPI Inference API

```
uvicorn src.app:app --host 0.0.0.0 --port 8000
```

The server starts at `http://localhost:8000`. Interactive docs are available at `http://localhost:8000/docs`.

3.1 Health Check

```
curl -X GET http://localhost:8000/health \
     -H "X-From: deployment-guide"
```

Expected response:

```
{"status": "ok", "model_loaded": true, "preprocessor_loaded": true}
```

3.2 Single Prediction

```
curl -X POST http://localhost:8000/predict \
     -H "Content-Type: application/json" \
     -H "X-From: deployment-guide" \
     -d '{
       "duration": 0, "protocol_type": "tcp", "service": "http", "flag": "S0",
       "src_bytes": 0, "dst_bytes": 0, "land": 0, "wrong_fragment": 0,
       "urgent": 0, "hot": 0, "num_failed_logins": 0, "logged_in": 0,
       "num_compromised": 0, "root_shell": 0, "su_attempted": 0,
       "num_root": 0, "num_file_creations": 0, "num_shells": 0,
       "num_access_files": 0, "num_outbound_cmds": 0,
       "is_host_login": 0, "is_guest_login": 0,
       "count": 511, "srv_count": 511,
       "serror_rate": 1.0, "srv_serror_rate": 1.0,
       "rerror_rate": 0.0, "srv_rerror_rate": 0.0,
       "same_srv_rate": 1.0, "diff_srv_rate": 0.0,
       "srv_diff_host_rate": 0.0, "dst_host_count": 255,
       "dst_host_srv_count": 255, "dst_host_same_srv_rate": 1.0,
       "dst_host_diff_srv_rate": 0.0, "dst_host_same_src_port_rate": 1.0,
       "dst_host_srv_diff_host_rate": 0.0, "dst_host_serror_rate": 1.0,
       "dst_host_srv_serror_rate": 1.0, "dst_host_rerror_rate": 0.0,
       "dst_host_srv_rerror_rate": 0.0
     }'
```

Expected response (DoS — Neptune attack):

```
{"prediction": 1, "label": "attack", "probability": 0.9981}
```

3.3 Batch Prediction

```
curl -X POST http://localhost:8000/predict/batch \
     -H "Content-Type: application/json" \
     -H "X-From: deployment-guide" \
     -d '{"records": [<record_1>, <record_2>]}'
```

> **Note:** `X-From` header is required on all requests for traceability. Requests without it return HTTP 400.

---

4. Running the Streamlit UI

```
streamlit run src/ui.py
```

Opens at `http://localhost:8501`.

Features:

- Preset traffic scenarios (Normal HTTP, Port Scan, DoS Attack)
- Full 41-feature input form
- Real-time prediction with attack probability gauge
- 🤖 AI Analyst Explanation (requires `OPENAI_API_KEY` — see step 5)

---

5. GenAI Analyst Explanations (Optional)

Two backends are supported — no OpenAI account required if you run a local LLM.

Option A: Local LLM (Ollama — free, runs offline)

```
# 1. Install Ollama: https://ollama.com
brew install ollama          # macOS

# 2. Pull a model (llama3.2 is fast and fits in 8 GB RAM)
ollama pull llama3.2

# 3. Start the server (OpenAI-compatible API on port 11434)
ollama serve

# 4. Configure .env
cp .env.example .env
# Set: LOCAL_LLM_URL=http://localhost:11434/v1
#      LOCAL_LLM_MODEL=llama3.2
```

Other local servers (LM Studio, llama.cpp) work the same way — set `LOCAL_LLM_URL` to their `/v1` endpoint.

Option B: OpenAI API

```
cp .env.example .env
# Set: OPENAI_API_KEY=sk-...
```

Restart the Streamlit UI after editing `.env`. The **"🤖 AI Analyst Explanation"** panel will generate plain-English SOC analyst briefings after each prediction. If neither variable is set the panel shows a graceful fallback message.

---

6. Retraining the Model

Run `train_and_save.py` at any time to retrain on updated data. The script overwrites `models/xgboost.pkl` and `models/preprocessor.pkl`.

```
# Retrain on the default NSL-KDD data
python train_and_save.py

# Recommended: point to a newer dataset (e.g. CICIDS-2017) by placing
# the CSV in data/raw/ and updating src/data_loader.py accordingly.
```

The running FastAPI server picks up new artefacts automatically on its next restart:

```
# Graceful restart (Uvicorn reloads on file change in dev mode)
uvicorn src.app:app --host 0.0.0.0 --port 8000 --reload
```

---

7. MLOps Practices

7.1 Model Versioning

All hyperparameters and the `best_model` identifier are persisted in `models/configs.yaml` at the end of every training run. Before retraining in production, snapshot the current artefacts:

```
# Snapshot before retraining
cp -r models/ models_backup_$(date +%Y%m%d)/
python train_and_save.py
```

7.2 Reproducibility

Practice	Implementation
Fixed random seed	`random_seed: 42` in `configs.yaml`
Pinned dependencies	`requirements.txt` with `>=` lower bounds
Saved preprocessor	`models/preprocessor.pkl` ensures identical transforms at inference
Saved processed splits	`data/processed/*.npy` for notebook reproducibility

7.3 Monitoring & Drift Detection

The `/health` endpoint provides a liveness signal suitable for uptime monitors (e.g. UptimeRobot, Prometheus blackbox exporter).

For **data drift detection** in production, integrate Evidently AI:

```
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset
import pandas as pd

report = Report(metrics=[DataDriftPreset()])
report.run(reference_data=reference_df, current_data=production_df)
report.save_html("reports/drift_report.html")
```

Recommended retraining trigger: retrain when the dataset drift score exceeds 0.1 on any of the top-10 SHAP features, or quarterly — whichever comes first.

7.4 Logging

The FastAPI app uses Python's standard `logging` module at `INFO` level. All prediction requests log the `X-From` header for auditability. In production, pipe logs to a SIEM:

```
uvicorn src.app:app --port 8000 2>&1 | tee -a logs/api.log
```

7.5 Security Controls

Control	Implementation
Input validation	Pydantic with range constraints on all 41 features
Injection prevention	No raw SQL or shell execution anywhere in the stack
Request traceability	`X-From` header required on every API call
Secret management	`OPENAI_API_KEY` via `.env` / environment — never hardcoded
Payload isolation	Model operates on metadata only; no network payload content read

---

8. Demo

> **Recording the demo (GIF/screencast):**
>
> 1. Start the Streamlit UI: `streamlit run src/ui.py`
> 2. Select the **"DoS Attack (Neptune)"** preset from the sidebar
> 3. Click **"🔍 Analyse Traffic"**
> 4. Show the ⚠️ ATTACK DETECTED result and probability gauge
> 5. Expand **"🤖 AI Analyst Explanation"** to show the LLM briefing
> 6. Switch preset to **"Normal HTTP Traffic"** and repeat
>
> Recommended tool: Kap (macOS) or ShareX (Windows) for GIF export.

Place the recorded file at `reports/demo.gif` and link it in `README.md`:

```
![Demo](reports/demo.gif)
```

---

10. Model Versioning & Rollback

Versioning Strategy

Each training run produces a timestamped backup before overwriting:

```
# Snapshot current artefacts before retraining
cp -r models/ models_backup_$(date +%Y%m%d_%H%M%S)/

# Retrain with new data
python train_and_save.py

# Verify new model performance before promoting
python -c "
import joblib, numpy as np
from sklearn.metrics import f1_score
X = np.load('data/processed/X_test.npy')
y = np.load('data/processed/y_test.npy')
m = joblib.load('models/xgboost.pkl')
print(f'New model F1: {f1_score(y, m.predict(X)):.4f}')
"

# Restart API to pick up new artefacts
# (zero-downtime: start new process, then stop old one)
uvicorn src.app:app --host 0.0.0.0 --port 8001 &  # new
# kill old process on port 8000
```

Rollback Procedure

```
# Restore previous model if new model degrades
cp models_backup_YYYYMMDD_HHMMSS/xgboost.pkl models/xgboost.pkl
cp models_backup_YYYYMMDD_HHMMSS/preprocessor.pkl models/preprocessor.pkl
# Restart API
```

Production Recommendation

Use DVC (Data Version Control) to version large binary artefacts with Git-compatible SHA tracking.

---

11. Monitoring Plan

Runtime Health

Check	Tool	Trigger
Liveness	`GET /health` → Prometheus blackbox	Alert if down > 1 min
Latency	Uvicorn access logs	Alert if p99 > 500ms
Error rate	HTTP 4xx/5xx rate	Alert if > 1%

Model Performance Drift

```
# Weekly batch job using Evidently AI
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset

report = Report(metrics=[DataDriftPreset()])
report.run(reference_data=reference_df, current_data=production_df)
report.save_html("reports/drift_report.html")
# Trigger retraining if drift score > 0.1 on top-10 SHAP features
```

Retraining trigger: drift score > 0.10 on any of: `serror_rate`, `same_srv_rate`, `src_bytes`, `dst_bytes`, `flag_S0`, `logged_in`

Retraining schedule: Quarterly minimum; immediately on drift alert.

---

12. CI/CD Pipeline

A GitHub Actions workflow (`.github/workflows/ci.yml`) runs on every push:

1. **Lint** — flake8 on `src/` and `tests/` (syntax errors = fail; style = warn)
2. **Train** — fast-mode `python train_and_save.py`
3. **Test** — `pytest tests/` (37 unit + API tests)
4. **Docker build** — validates the container image builds

---

9. Project Structure Reference

```
capstone/
├── src/
│   ├── app.py                  # FastAPI REST API
│   ├── ui.py                   # Streamlit interactive UI
│   ├── genai_explainer.py      # LLM analyst explanation module
│   ├── models.py               # Model training + evaluation
│   ├── preprocessor.py         # Feature pipeline
│   ├── data_loader.py          # NSL-KDD loader
│   ├── feature_engineering.py  # Domain features + PCA
│   ├── explainability.py       # SHAP + LIME
│   └── bias_audit.py           # Fairness audit
├── models/
│   ├── xgboost.pkl             # Best model artefact
│   ├── preprocessor.pkl        # Fitted transformer
│   └── configs.yaml            # Hyperparameters + metadata
├── train_and_save.py           # One-shot training pipeline
├── requirements.txt
├── .env.example                # Environment variable template
└── DEPLOYMENT.md               # This file
```